Автономная некоммерческая образовательная организация высшего образования

«Научно-технологический университет «Сириус»»

Лабораторная работа № 3

«CRC-8 сервер по UART»

По дисциплине: Основы программирования для микроэлектроники

Выполнил:

Студент

Мегерян Сергей Сергеевич

Проверил:

_____

ФТ Сириус, 2026 г.

Содержание

Тема работы

Цель работы

Постановка задачи

Теоретические сведения

- Что такое CRC
- Порождающий полином
- Принцип вычисления CRC-8

Аппаратное и программное обеспечение

Схема подключения

Описание работы программы

Проверка базовой версии со сбросом

- Эксперимент 1. Передача одного символа A
- Эксперимент 2. Передача строки AAA

Почему при передаче AAA CRC меняется

Проверка измененной версии без сброса

Сравнение двух режимов работы

- Версия со сбросом
- Версия без сброса

Вывод

Тема работы

Реализация сервера на базе Arduino Uno, выполняющего вычисление контрольной суммы CRC-8 для сообщений, принимаемых с ПК по интерфейсу UART, с выдачей результата по внешнему сигналу.

Цель работы

Изучить принципы построения циклического избыточного кода CRC-8, реализовать прием данных по последовательному интерфейсу UART, организовать выдачу результата по нажатию кнопки и сравнить два режима работы программы: со сбросом накопленного остатка и без сброса.

Постановка задачи

По техническому заданию требовалось:

1. реализовать сервер на базе Arduino, принимающий сообщения от ПК по UART;
2. вычислять для принятых данных контрольную сумму CRC-8 по заданному порождающему полиному;

3. выдавать рассчитанное значение по внешнему сигналу, формируемому тактовой кнопкой;

4. использовать скорость обмена 9600 бод;

5. подключить кнопку в режиме внутренней подтяжки INPUT_PULLUP;

6. провести проверку для одного символа A и для строки AAA;

7. объяснить, почему при изменении длины сообщения изменяется CRC;

8. модифицировать программу так, чтобы после нажатия кнопки контрольная сумма не обнулялась, а продолжала накапливаться.

Теоретические сведения

Что такое CRC

CRC (Cyclic Redundancy Check) — это циклический избыточный код, применяемый для контроля целостности данных при передаче и хранении. Метод позволяет обнаруживать случайные ошибки в сообщениях за счет вычисления остатка от деления двоичного полинома сообщения на заранее выбранный порождающий полином.

На практике CRC широко используется:

- в последовательных интерфейсах передачи данных;
- в сетевых протоколах;

- в системах хранения информации;
- в промышленных и встроенных устройствах.

Главное назначение CRC — обнаружение искажений данных без существенного увеличения объема передаваемой информации.

Порождающий полином

В данной работе используется стандартный порождающий полином CRC-8:

$G(x) = x^8 + x^2 + x + 1$

В шестнадцатеричной записи в программе он задается как:

0x07

Старший член x^8 в программной записи не хранится явно, поэтому в коде остается только младшая часть полинома. Именно с этим полиномом выполняется побитовая обработка каждого входного байта.

Принцип вычисления CRC-8

Алгоритм вычисления реализован побитово и состоит из следующих шагов:

2. очередной принятый байт включается в текущий остаток CRC операцией XOR;

2. далее выполняются восемь циклов обработки — по числу бит в байте;

3. если старший бит остатка равен 1, остаток сдвигается влево и затем выполняется XOR с полиномом 0x07;

4. если старший бит равен 0, выполняется только сдвиг влево;

5. после обработки всех принятых байтов формируется итоговая контрольная сумма.

С математической точки зрения это эквивалентно делению двоичного полинома сообщения на порождающий полином по модулю 2.

Аппаратное и программное обеспечение

В работе использовались:

- плата Arduino Uno;
- одна тактовая кнопка;
- макетная плата и соединительные провода;
- персональный компьютер с Arduino IDE;

- USB-кабель для связи с ПК.

Схема подключения

Схема включает только кнопку и подключение платы к компьютеру:

- D2 -> кнопка -> GND;
- USB Arduino -> ПК.

Вход D2 настроен в режиме INPUT_PULLUP, поэтому при отпущенной кнопке на входе поддерживается логическая единица, а при нажатии вход замыкается на землю и фиксируется логический ноль.

Описание работы программы

В программе Arduino Uno функционирует как сервер, принимающий байты от ПК по интерфейсу UART со скоростью 9600 бод. Каждый принятый байт немедленно включается в процесс расчета CRC-8. При нажатии кнопки, подключенной к выводу D2, текущее значение контрольной суммы выводится в Serial Monitor.

В репозитории подготовлены две версии программы:

- lab3_crc8_reset/lab3_crc8_reset.ino — версия, в которой после вывода результата аккумулятор CRC сбрасывается;
- lab3_crc8_accumulate/lab3_crc8_accumulate.ino — версия, в которой после вывода результата состояние аккумулятора сохраняется и дальнейшие сообщения продолжают накапливаться.

Проверка базовой версии со сбросом

Для проверки использовался Serial Monitor со скоростью 9600 бод в режиме No line ending.

Эксперимент 1. Передача одного символа A

На вход передается один байт:

0x41

После нажатия кнопки программа выдает:

CRC-8 = 0xC0

Эксперимент 2. Передача строки AAA

После сброса аккумулятора передаются три байта:

0x41 0x41 0x41

После нажатия кнопки программа выдает:

CRC-8 = 0x63

Почему при передаче AAA CRC меняется

Контрольная сумма CRC зависит не только от значения отдельных символов, но и от длины сообщения, порядка байтов и промежуточного остатка после каждого шага обработки. После вычисления CRC для первого символа A формируется определенный остаток. Когда затем принимаются еще два символа A, дальнейшая обработка уже выполняется не с нулевого состояния, а с полученным промежуточным значением. Поэтому итоговая сумма для AAA отличается от суммы для одного символа A.

Следовательно, CRC(A) и CRC(AAA) относятся к двум разным сообщениям и не обязаны совпадать.

Проверка измененной версии без сброса

Во второй версии программы после нажатия кнопки аккумулятор CRC не обнуляется.

Если:

3. отправить A;

2. нажать кнопку;

3. затем отправить AAA;

4. снова нажать кнопку,

то итоговая контрольная сумма будет вычисляться уже не только для новой строки AAA, а для всей накопленной последовательности AAAA.

В этом случае ожидаемое значение:

CRC-8 = 0xEE

Сравнение двух режимов работы

Версия со сбросом

- удобна для независимой проверки отдельных сообщений;
- после каждого нажатия кнопки начинается новое вычисление;
- хорошо подходит для дискретных запросов.

Версия без сброса

- сохраняет промежуточное состояние аккумулятора;
- позволяет вести непрерывный подсчет CRC для потока сообщений;
- соответствует задаче модификации программы по техническому заданию.

Вывод

В ходе лабораторной работы был реализован сервер на базе Arduino Uno, принимающий данные по UART и вычисляющий контрольную сумму CRC-8 по заданному порождающему полиному 0x07. Было показано, что для одного символа A получается CRC-8 = 0xC0, а для строки AAA — CRC-8 = 0x63, что объясняется зависимостью алгоритма от длины и структуры сообщения. Также была разработана модифицированная версия программы, в которой контрольная сумма после нажатия кнопки не сбрасывается и продолжает накапливаться для последующих сообщений.

Таблица соединений

Элемент	Подключение

Кнопка выдачи CRC	D2 -> кнопка -> GND
UART	USB-подключение Arduino к ПК
Скорость	9600 бод
Режим отправки	No line ending

Иллюстрации и скриншоты

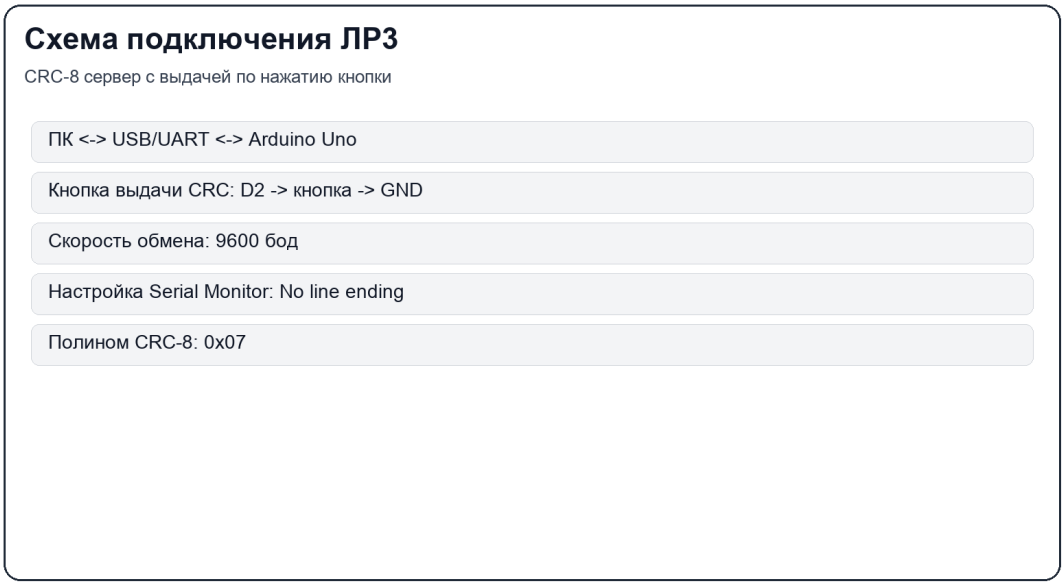

Рисунок 1 - Структурная схема подключения CRC-8 сервера

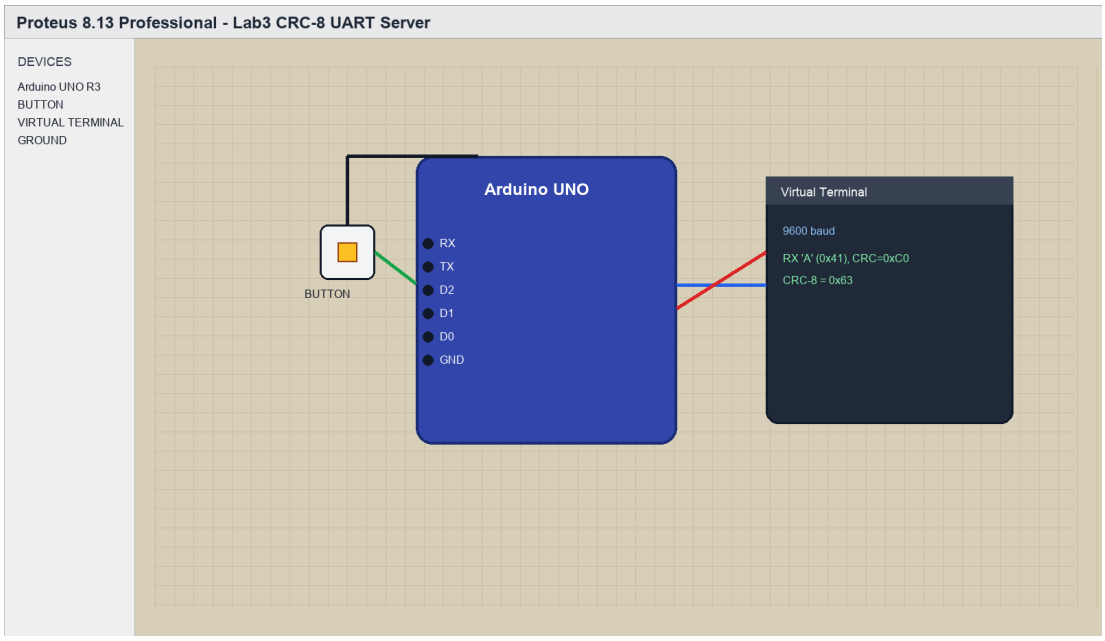

Рисунок 2 - Внешний вид модели CRC-8 сервера в Proteus 8.13

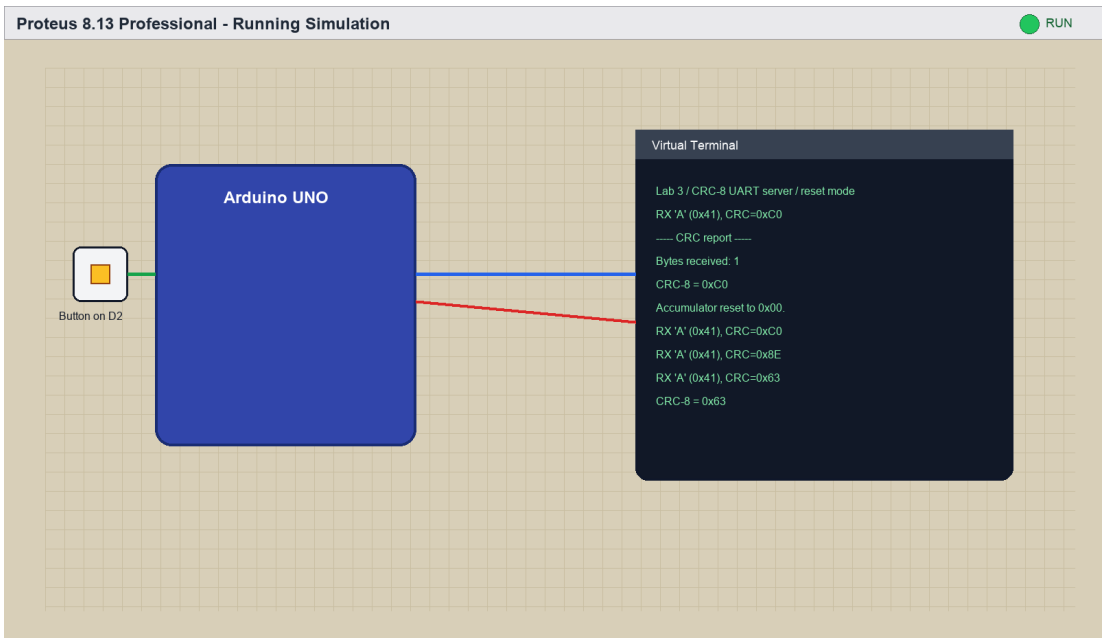

Рисунок 3 - Модель в Proteus в процессе выполнения

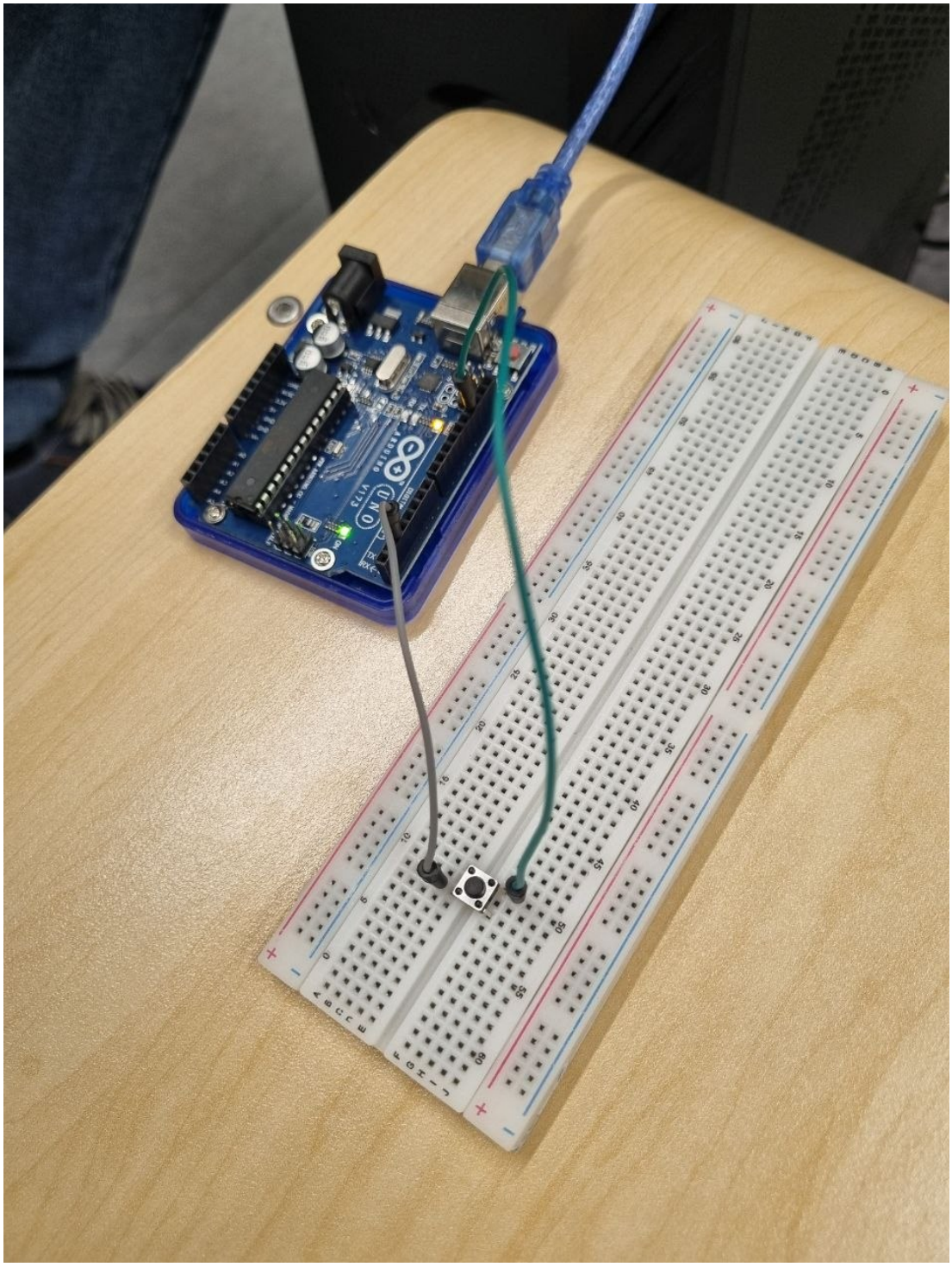

Рисунок 4 - Реальное подключение кнопки к Arduino Uno

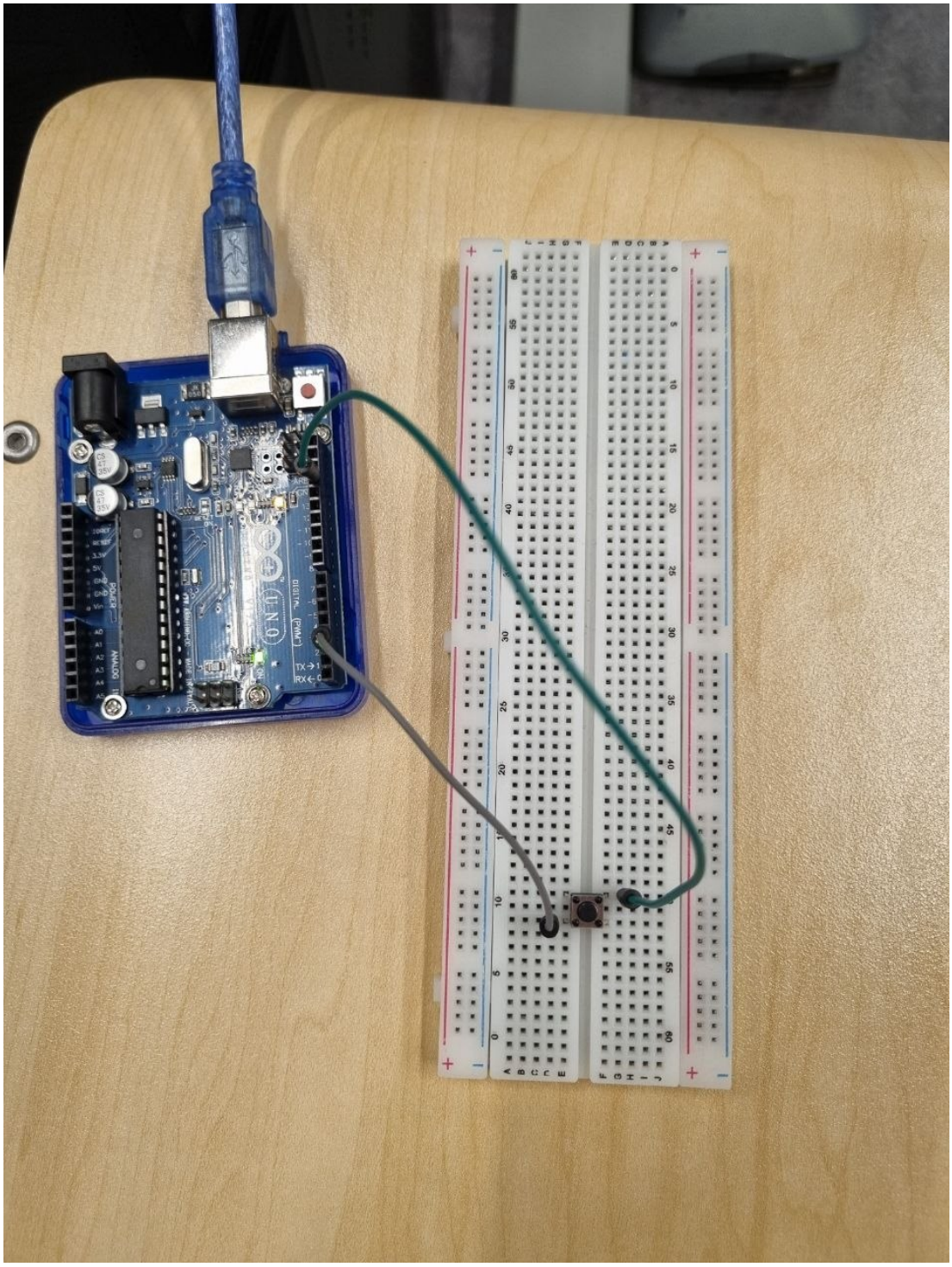

Рисунок 5 - Собранный стенд лабораторной работы

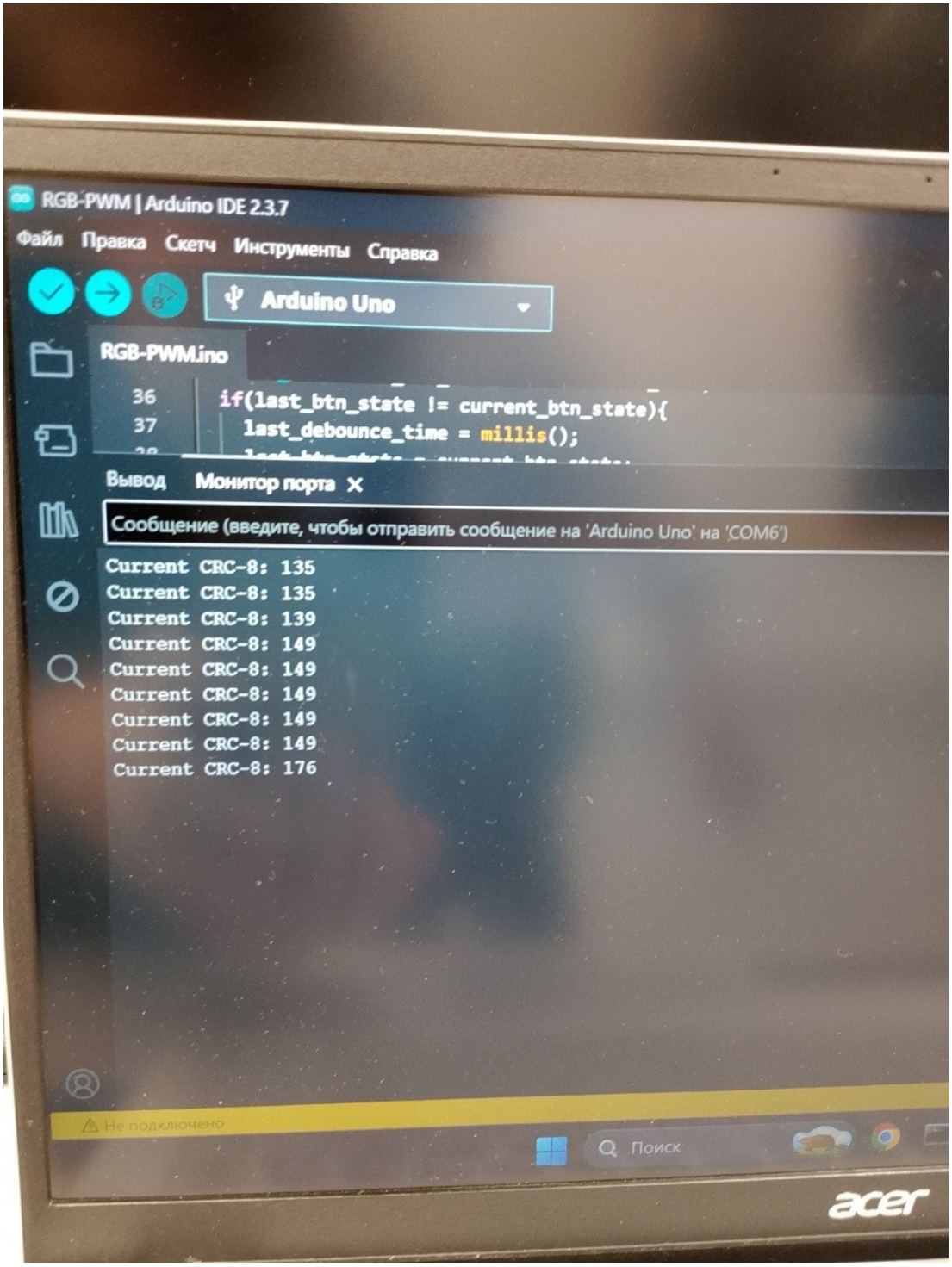

Рисунок 6 - Фото вывода результатов в Serial Monitor

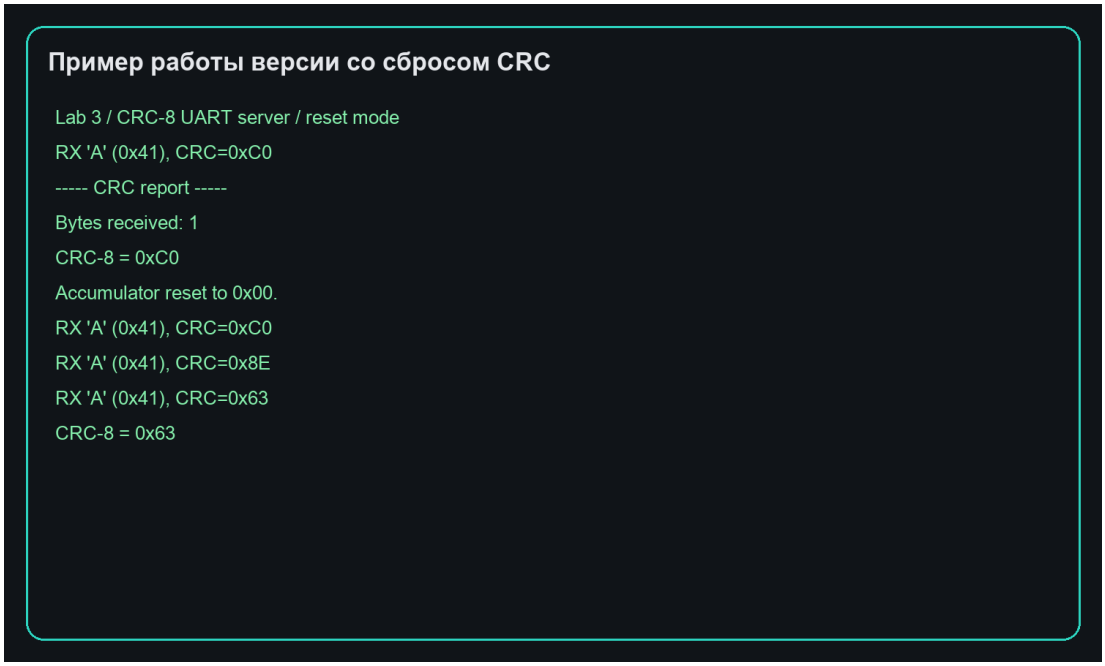

Рисунок 7 - Пример работы версии со сбросом CRC

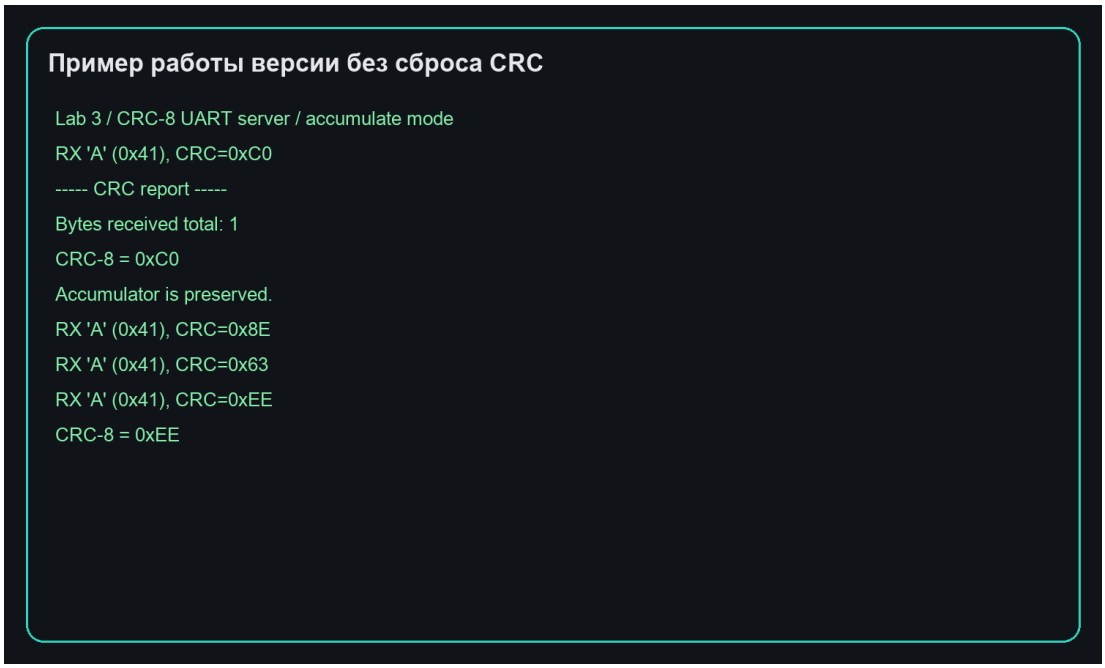

Рисунок 8 - Пример работы версии без сброса CRC

Приложение. Листинг программы

lab3_crc8_reset.ino

```
#include <Arduino.h>
```

```
const uint8_t REPORT_BUTTON_PIN = 2;
const uint8_t CRC_POLYNOMIAL = 0x07;  // x^8 + x^2 + x + 1
const unsigned long SERIAL_BAUDRATE = 9600;
const unsigned long DEBOUNCE_DELAY_MS = 50;

uint8_t currentCrc = 0x00;
unsigned long bytesReceived = 0;

bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;

uint8_t updateCrc8(uint8_t crc, uint8_t dataByte) {
  crc ^= dataByte;

  for (uint8_t bit = 0; bit < 8; bit++) {
    if (crc & 0x80) {
      crc = static_cast<uint8_t>((crc << 1) ^ CRC_POLYNOMIAL);
    } else {
      crc <<= 1;
    }
  }

  return crc;
}

void printHexByte(uint8_t value) {
  if (value < 0x10) {
    Serial.print('0');
  }
  Serial.print(value, HEX);
}

void printByteDescription(uint8_t value) {
  if (value >= 32 && value <= 126) {
    Serial.print('\'');
    Serial.print(static_cast<char>(value));
    Serial.print('\'');
  } else {
```

```
    Serial.print(F("non-printable"));
  }
}


void handleIncomingUart() {
  while (Serial.available() > 0) {
    uint8_t receivedByte = static_cast<uint8_t>(Serial.read());
    currentCrc = updateCrc8(currentCrc, receivedByte);
    bytesReceived++;


    Serial.print(F("RX "));
    printByteDescription(receivedByte);
    Serial.print(F(" (0x"));
    printHexByte(receivedByte);
    Serial.print(F("), CRC=0x"));
    printHexByte(currentCrc);
    Serial.println();
  }
}


void printAndResetCrc() {
  Serial.println(F("----- CRC report -----"));
  Serial.print(F("Bytes received: "));
  Serial.println(bytesReceived);
  Serial.print(F("CRC-8 = 0x"));
  printHexByte(currentCrc);
  Serial.println();
  Serial.println(F("Accumulator reset to 0x00."));
  Serial.println(F("----------------------"));


  currentCrc = 0x00;
  bytesReceived = 0;
}


void handleReportButton() {
  bool reading = digitalRead(REPORT_BUTTON_PIN);


  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
    lastButtonReading = reading;
  }
```

```
    if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS && reading !=
stableButtonState) {
      stableButtonState = reading;

      if (stableButtonState == LOW) {
        printAndResetCrc();
      }
    }
  }

  void setup() {
    pinMode(REPORT_BUTTON_PIN, INPUT_PULLUP);
    Serial.begin(SERIAL_BAUDRATE);

    Serial.println(F("Lab 3 / CRC-8 UART server / reset mode"));
    Serial.println(F("Polynomial: 0x07 (x^8 + x^2 + x + 1)"));
    Serial.println(F("UART speed: 9600"));
    Serial.println(F("Serial Monitor setting: No line ending"));
    Serial.println(F("Press the button on D2 to print CRC and reset the
accumulator."));
  }

  void loop() {
    handleIncomingUart();
    handleReportButton();
  }
```

lab3_crc8_accumulate.ino

```
  #include <Arduino.h>

  const uint8_t REPORT_BUTTON_PIN = 2;
  const uint8_t CRC_POLYNOMIAL = 0x07;  // x^8 + x^2 + x + 1
  const unsigned long SERIAL_BAUDRATE = 9600;
  const unsigned long DEBOUNCE_DELAY_MS = 50;

  uint8_t currentCrc = 0x00;
  unsigned long bytesReceived = 0;
```

```
bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;


uint8_t updateCrc8(uint8_t crc, uint8_t dataByte) {
  crc ^= dataByte;


  for (uint8_t bit = 0; bit < 8; bit++) {
    if (crc & 0x80) {
      crc = static_cast<uint8_t>((crc << 1) ^ CRC_POLYNOMIAL);
    } else {
      crc <<= 1;
    }
  }


  return crc;
}


void printHexByte(uint8_t value) {
  if (value < 0x10) {
    Serial.print('0');
  }
  Serial.print(value, HEX);
}


void printByteDescription(uint8_t value) {
  if (value >= 32 && value <= 126) {
    Serial.print('\'');
    Serial.print(static_cast<char>(value));
    Serial.print('\'');
  } else {
    Serial.print(F("non-printable"));
  }
}


void handleIncomingUart() {
  while (Serial.available() > 0) {
    uint8_t receivedByte = static_cast<uint8_t>(Serial.read());
    currentCrc = updateCrc8(currentCrc, receivedByte);
    bytesReceived++;
```

```
      Serial.print(F("RX "));
      printByteDescription(receivedByte);
      Serial.print(F(" (0x"));
      printHexByte(receivedByte);
      Serial.print(F("), CRC=0x"));
      printHexByte(currentCrc);
      Serial.println();
    }
  }

  void printWithoutReset() {
    Serial.println(F("----- CRC report -----"));
    Serial.print(F("Bytes received total: "));
    Serial.println(bytesReceived);
    Serial.print(F("CRC-8 = 0x"));
    printHexByte(currentCrc);
    Serial.println();
    Serial.println(F("Accumulator is preserved."));
    Serial.println(F("----------------------"));
  }

  void handleReportButton() {
    bool reading = digitalRead(REPORT_BUTTON_PIN);

    if (reading != lastButtonReading) {
      lastDebounceTime = millis();
      lastButtonReading = reading;
    }

    if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS && reading !=
stableButtonState) {
      stableButtonState = reading;

      if (stableButtonState == LOW) {
        printWithoutReset();
      }
    }
  }

  void setup() {
```

```
  pinMode(REPORT_BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(SERIAL_BAUDRATE);


  Serial.println(F("Lab 3 / CRC-8 UART server / accumulate mode"));
  Serial.println(F("Polynomial: 0x07 (x^8 + x^2 + x + 1)"));
  Serial.println(F("UART speed: 9600"));
  Serial.println(F("Serial Monitor setting: No line ending"));
  Serial.println(F("Press the button on D2 to print CRC without resetting the
accumulator."));
}


void loop() {
  handleIncomingUart();
  handleReportButton();
}
```